

Система инициализации SystemV.

Введение

В UNIX-подобных операционных системах процесс загрузки представляет собой строгую последовательность этапов, завершающихся передачей управления ядру операционной системы. После инициализации оборудования и базовых структур памяти ядро запускает первый пользовательский процесс, который получает идентификатор PID 1 (Process ID 1)[1].

Данный процесс носит название `init`. Его главная задача - инициализация пользовательского пространства, запуск системных служб (демонов), монтирование файловых систем и управление жизненным циклом других процессов. Стандарт SystemV `init` на протяжении нескольких десятилетий являлся доминирующей архитектурой выполнения данной задачи в серверных и настольных UNIX-системах.

История

До появления архитектуры SystemV стандартом де-факто являлся BSD `init`. Его механизм работы отличался простотой: после загрузки ядра система выполняла единый монолитный скрипт инициализации (`/etc/rc`). Данный подход удовлетворял базовые потребности, однако с ростом сложности серверной инфраструктуры администраторам потребовалась большая гибкость — в частности, возможность переводить систему в различные режимы обслуживания без полной перезагрузки[2].

В 1983 году корпорация AT&T представила коммерческий выпуск операционной системы UNIX SystemV[1]. Ключевым нововведением релиза стала модульная система инициализации, базирующаяся на концепции «уровней выполнения» (Runlevels).

Этот архитектурный подход продемонстрировал высокую надежность и масштабируемость, вследствие чего был адаптирован большинством разработчиков операционных систем, включая коммерческие версии (Solaris, HP-UX) и открытые платформы (дистрибутивы GNU/Linux, где реализация получила название `sysvinit`).

Архитектура

Основой логики SystemV является использование уровней выполнения. Уровень выполнения — это строго определенное состояние операционной системы, характеризующееся набором запущенных служб и доступных функций[3].

Спецификация SystemV

Уровень	Название	Характеристика
0	Half	Остановка системы. Переход на данный уровень инициирует корректное завершение всех процессов и отключение питания.
1	Single-user mode	Однопользовательский (административный) режим. Сетевые службы отключены, доступ предоставляется исключительно суперпользователю (root). Применяется для восстановления и диагностики.
2	Multi-user (без сети)	Многопользовательский режим без поддержки сетевых протоколов (в некоторых дистрибутивах Linux сеть может быть активна).
3	Multi-user (с сетью)	Стандартный серверный режим. Полнофункциональный многопользовательский режим с поддержкой сети и запуском всех служб, но без графического интерфейса.
4	Undefined	Резервный уровень, доступный для пользовательской настройки (на практике применяется редко).
5	X11 / GUI	Режим рабочей станции. Идентичен уровню 3, но включает автоматический запуск графического сервера (X Window System) и менеджера входа в систему (Display Manager).
6	Reboot	Перезагрузка системы. Иницирует корректное завершение работы и последующий рестарт.

Конфигурация системы: структура файла /etc/inittab

Управление поведением процесса `init` осуществляется через центральный конфигурационный файл `/etc/inittab`. При загрузке системы процесс `init` считывает данный файл для определения уровня выполнения по умолчанию и действий, которые необходимо предпринять.

Каждая строка конфигурационного файла имеет строго регламентированный синтаксис:

`id : runlevels : action : process`

Синтаксис `/etc/inittab`

Поле	Назначение	Пример использования
id	Уникальный строковый идентификатор записи (от 1 до 4 символов).	id, si
runlevels	Перечень уровней выполнения, к которым применяется правило.	35
action	Директива, определяющая характер запуска процесса.	initdefault, respawn
process	Абсолютный путь к исполняемому файлу или скрипту с аргументами.	/sbin/mingetty tty1

Маршрутизация запуска: механизм RC-скриптов

Для управления системными службами `SystemV` использует иерархическую структуру директорий. Для каждого уровня выполнения существует собственная директория (например, `/etc/rc.d/rc3.d/` для уровня 3).

- Внутри данных директорий располагаются символические ссылки на основные скрипты управления службами (хранящиеся в `/etc/init.d/`). Имена ссылок имеют строгую номенклатуру:
 - Префикс `S` (`Start`) обозначает скрипты, которые необходимо запустить при входе на данный уровень.
 - Префикс `K` (`Kill`) обозначает скрипты служб, которые необходимо остановить.
 - За префиксом следует двузначное число (от 00 до 99), определяющее приоритет (порядок) выполнения.
- Примеры

- S10network — инициализация сетевых интерфейсов (ранний этап).
- S80httpd — запуск веб-сервера (поздний этап, требующий наличия активной сети).
- K15postgresql — остановка базы данных при выходе из уровня выполнения.

Инструментарий системного администратора

Ручное создание символических ссылок в SystemV считается плохой практикой. Для управления подсистемой используются высокоуровневые утилиты [5]:

- service: инкапсулирует вызов скриптов в чистой среде.
- chkconfig (Red Hat / CentOS): автоматизирует создание ссылок S/K (например, chkconfig httpd on).
- update-rc.d (Debian / Ubuntu): функциональный аналог для deb-ориентированных систем.

Эволюционный переход и преемники SystemV

По мере развития серверной инфраструктуры, появления облачных технологий и динамически подключаемых устройств (plug-and-play), фундаментальные ограничения SystemV стали очевидны. Строго последовательное выполнение скриптов приводило к неприемлемо долгой загрузке ОС, а отсутствие подсистемы мониторинга состояния демонов требовало написания сложных скриптов-костылей с PID-файлами[3].

Это привело к появлению альтернативных систем:

- Upstart (разработка Canonical): представила событийно-ориентированную модель, где службы запускались не по порядку, а в ответ на события (например, «подключен сетевой кабель» или «смонтирован диск»).
- systemd: современный стандарт индустрии. Полностью отказался от bash-скриптов в пользу декларативных конфигурационных файлов (unit-файлов). Обеспечивает агрессивное параллельное выполнение служб, строгий контроль процессов через механизмы ядра (cgroups) и управление зависимостями на уровне сокетов[4].

Преимущества и недостатки архитектуры SystemV

На протяжении многих лет SystemV оставалась эталоном стабильности, однако развитие аппаратного обеспечения выявило ряд системных ограничений данной архитектуры.

- Преимущества
 - Прозрачность логики: все конфигурации представляют собой текстовые файлы и bash-скрипты, доступные для аудита и ручного редактирования.
 - Предсказуемость: строгая последовательность выполнения скриптов исключает состояние «гонки» (race conditions) при загрузке.
 - Высокая отказоустойчивость: архитектура проверена десятилетиями промышленной эксплуатации на критически важных узлах[1].
- Недостатки
 - Низкая скорость инициализации: строго последовательный (синхронный) запуск служб не позволяет утилизировать вычислительные мощности многоядерных процессоров при загрузке.
 - Избыточность программного кода: скрипты инициализации (/etc/init.d/) часто содержат дублирующий код для обработки базовых функций (проверка статуса, чтение PID-файлов).
 - Отсутствие динамического управления: SystemV плохо адаптирована к современным аппаратным средам, предполагающим «горячее» подключение устройств (plug-and-play, съемные носители, динамические изменения конфигурации сети)[2].

Заключение

Архитектура SystemV init сыграла фундаментальную роль в эволюции операционных систем семейства UNIX. Концепция уровней выполнения и стандартизированного управления процессами заложила основы системного администрирования. Несмотря на то, что в наше время стандарт SystemV практически полностью вытеснен асинхронными системами нового поколения (такими как systemd), принципы его работы остаются обязательными для глубокого понимания архитектуры операционных систем и исторического контекста их развития[3].

Список литературы

@book{robachevsky2010unix,
author = {Робачевский, А. М. and Немнюгин, С. А. and Стесик, О. Л.},
title = {Операционная система UNIX},

edition = {2-е изд.},
address = {СПб.},
publisher = {БХВ-Петербург},
year = {2010},
language = {russian}
}

@book{irtegov2008os,
author = {Иртегов, Д. В.},
title = {Введение в операционные системы},
edition = {2-е изд.},
address = {СПб.},
publisher = {БХВ-Петербург},
year = {2008},
language = {russian}
}

@book{stolyarov2016os,
author = {Столяров, А. В.},
title = {Введение в операционные системы},
address = {М.},
publisher = {МАКС Пресс},
year = {2016},
language = {russian}
}

@book{kolisnichenko2021linux,
author = {Колисниченко, Д. Н.},
title = {Linux. От новичка к профессионалу},
edition = {7-е изд.},
address = {СПб.},
publisher = {БХВ-Петербург},
year = {2021},
language = {russian}
}

@book{yaremchuk2011linux,
author = {Яремчук, С. А.},
title = {Linux для системного администратора},
address = {СПб.},
publisher = {БХВ-Петербург},
year = {2011},

```
language = {russian}  
}
```